



CSE 4621: Microprocessors and Interfacing

Lecture 1 — Introduction to Microprocessors and Assembly Language

Syed Rifat Raiyan

Lecturer

Department of Computer Science & Engineering
Islamic University of Technology, Dhaka, Bangladesh

Email: rifatraiyan@iut-dhaka.edu

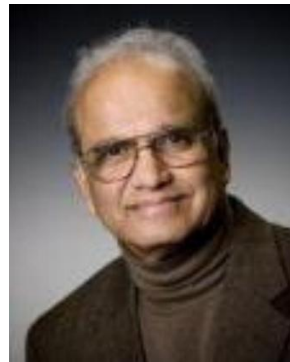
Introduction

First things first...

Google Classroom – m7v26wdm

Relevant Resources

- Books:
 - ✓ *Microprocessors and Interfacing: Programming and Hardware*, by Douglas V. Hall
 - ✓ *Microprocessor, architecture, programming & application with the 8085*, by Ramesh Gaonkar
 - ✓ *Assembly Language Programming and Organization of the IBM PC*, by Charles Marut and Ytha Y. Yu
 - ✓ *Intel Microprocessors Architecture, Programming, and Interfacing*, by Barry B. Brey



Keep an eye on the GC for the Slides and Lecture notes.

Introduction

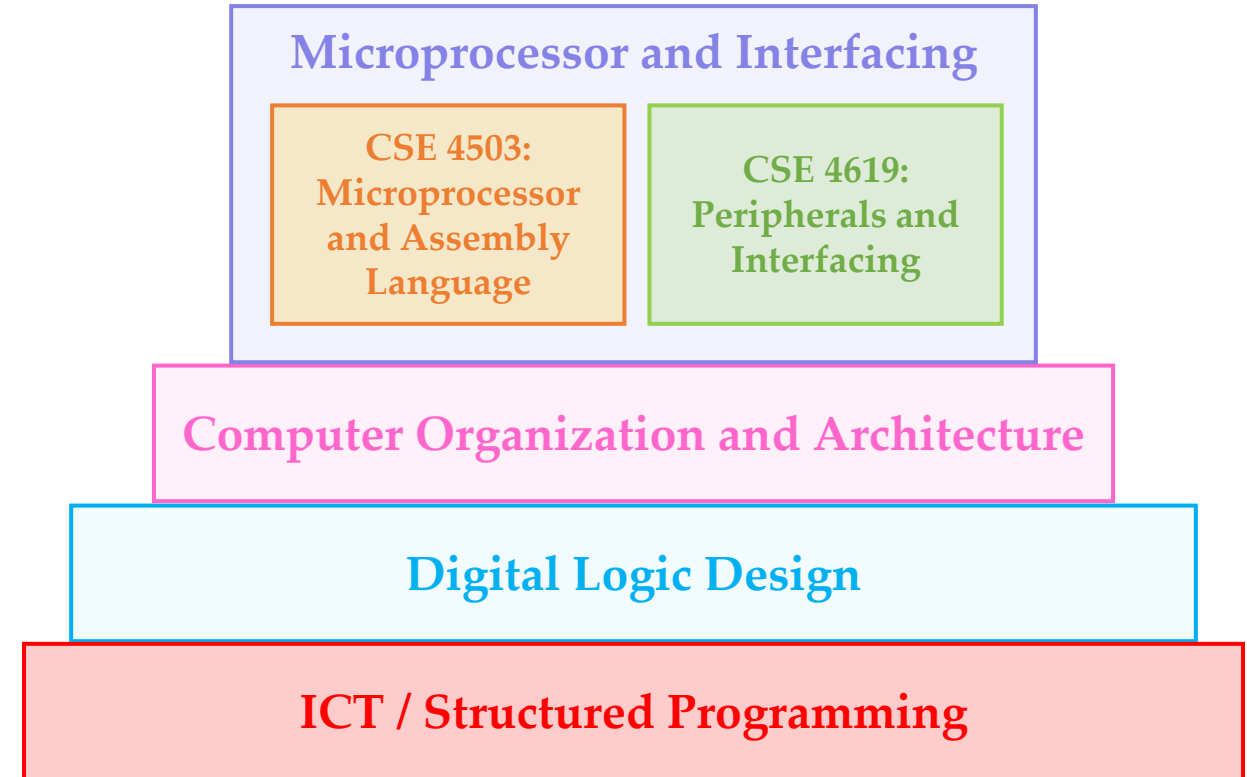
How to do well in this course?

For the **theory course** CSE 4621 —

- Pay attention to the class lectures.
- Take notes.
- Go through:
 - ✓ Lecture slides
 - ✓ Books (*optional*)
- Solve previous-year questions.

For the **lab course** CSE 4622 —

- Pay attention to the lab lectures and take notes.
- Go through the lab handouts.
- Practise simple coding tasks on **EMU8086**.



Lecture Plan

The agenda for today

- Primary Functions of a Computer
- Major Components of a Computer
- What is a Microprocessor?
- Chronological Evolution of Processors
- Assembly Language
- **.asm** program examples for Intel 8085 and 8086

Primary Functions

What does a computer do?

At the most fundamental level, computers do two main things:

Data Storage

A computer stores data and instructions so they can be used when needed.

- temporary storage in registers
- short-term storage in main memory
- long-term storage in secondary devices
- storage of intermediate and final results

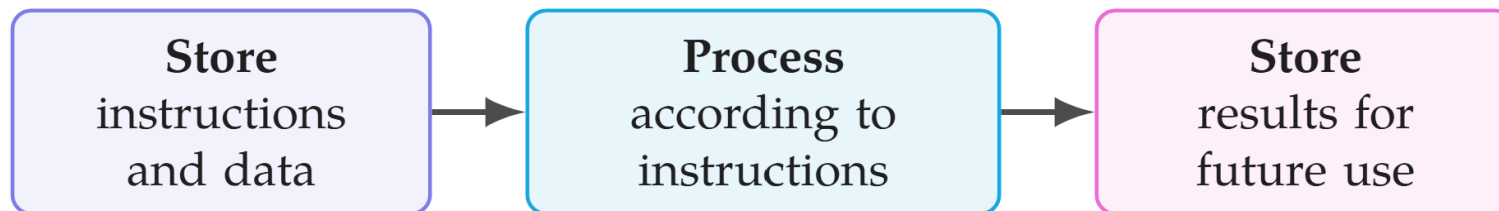
In this course, we mainly study how the processor accesses **memory locations** and exchanges data with the system.

Data Processing

A computer processes data by applying instructions to it.

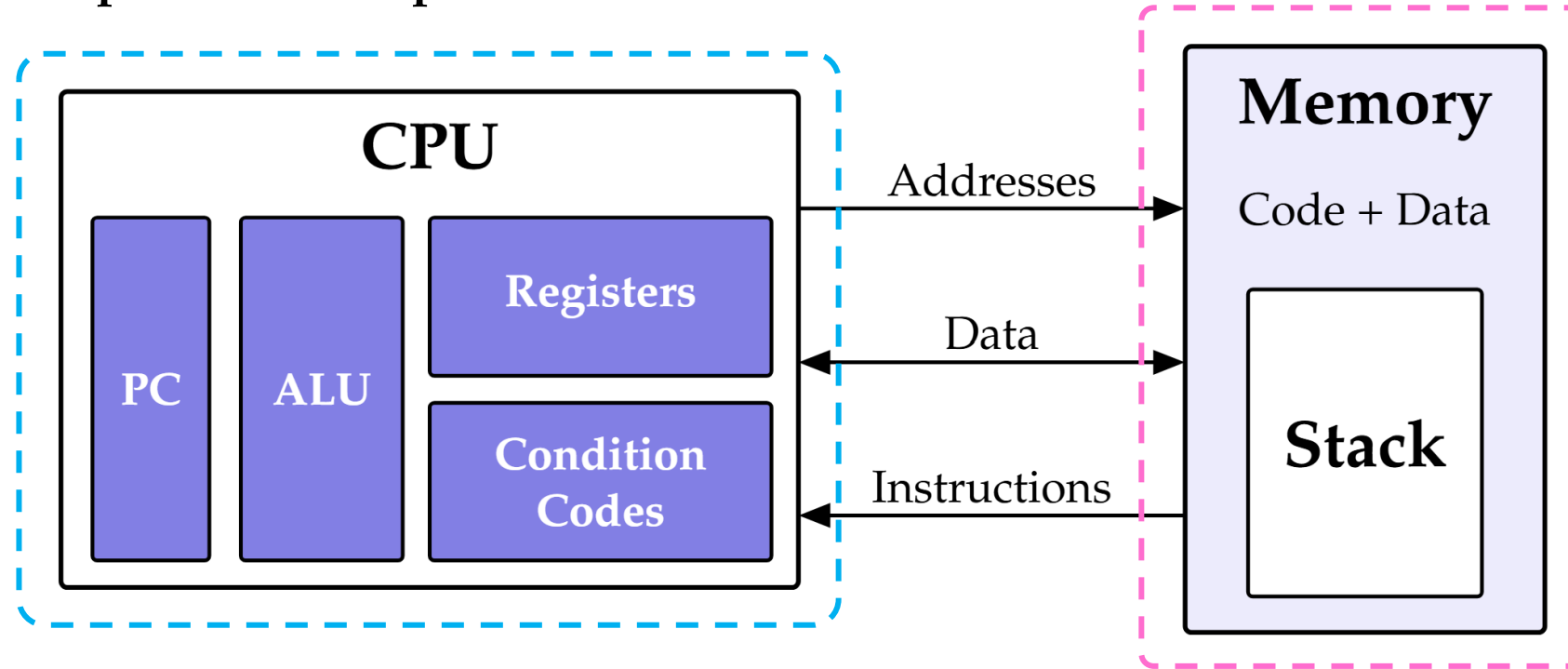
- arithmetic operations
- logical comparisons
- decision making and branching
- movement of data between registers and memory

In a microprocessor, this work is mainly done by the **ALU**, **control unit**, and **register set**.



Main Components

Breaking down the parts of a computer



System Overview

A computer can be viewed as two cooperating blocks:

- the **CPU**, which executes instructions
- the **memory**, which stores code, data, and the stack

The next steps explain how control and data move between them.

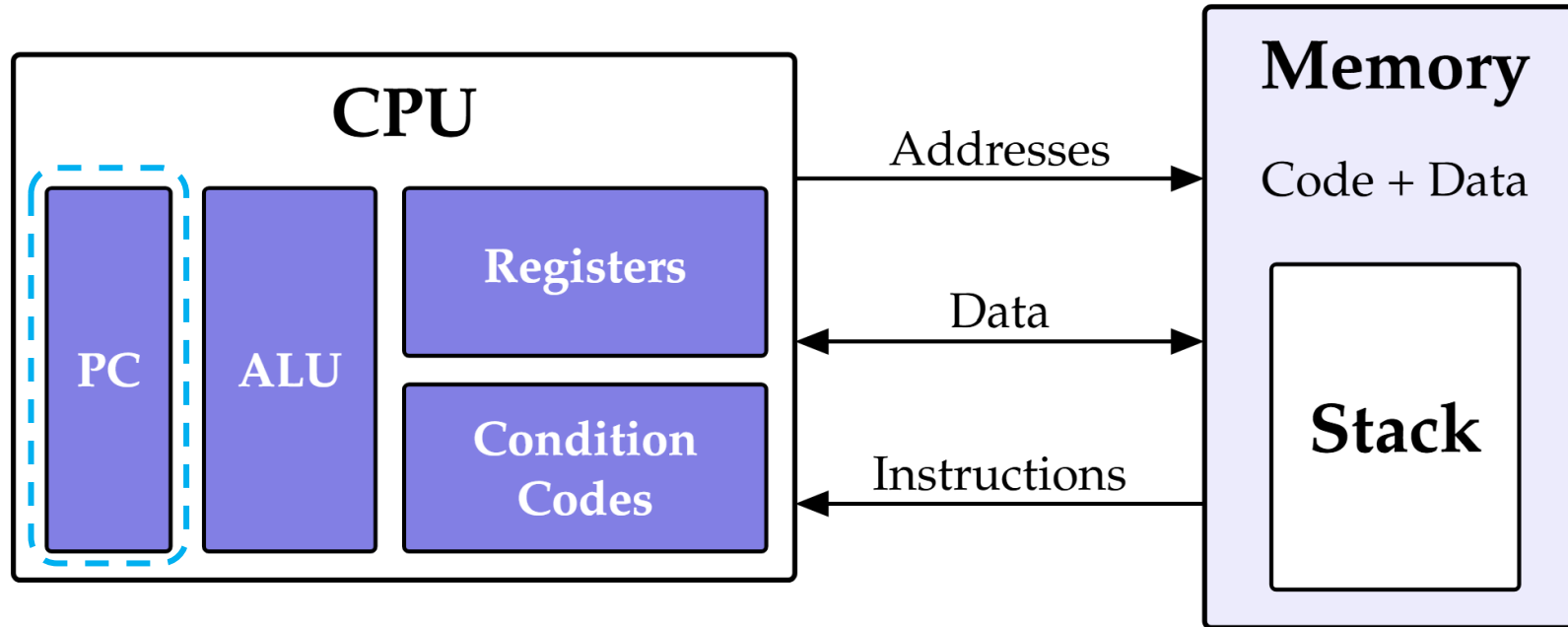
System Buses

Information exchange between CPU and memory is facilitated by:

- Address Bus (*Unidirectional*)
- Data Bus (*Bidirectional*)
- Control Bus (*Bidirectional*)

Main Components

Breaking down the parts of a computer



Program Counter (PC)

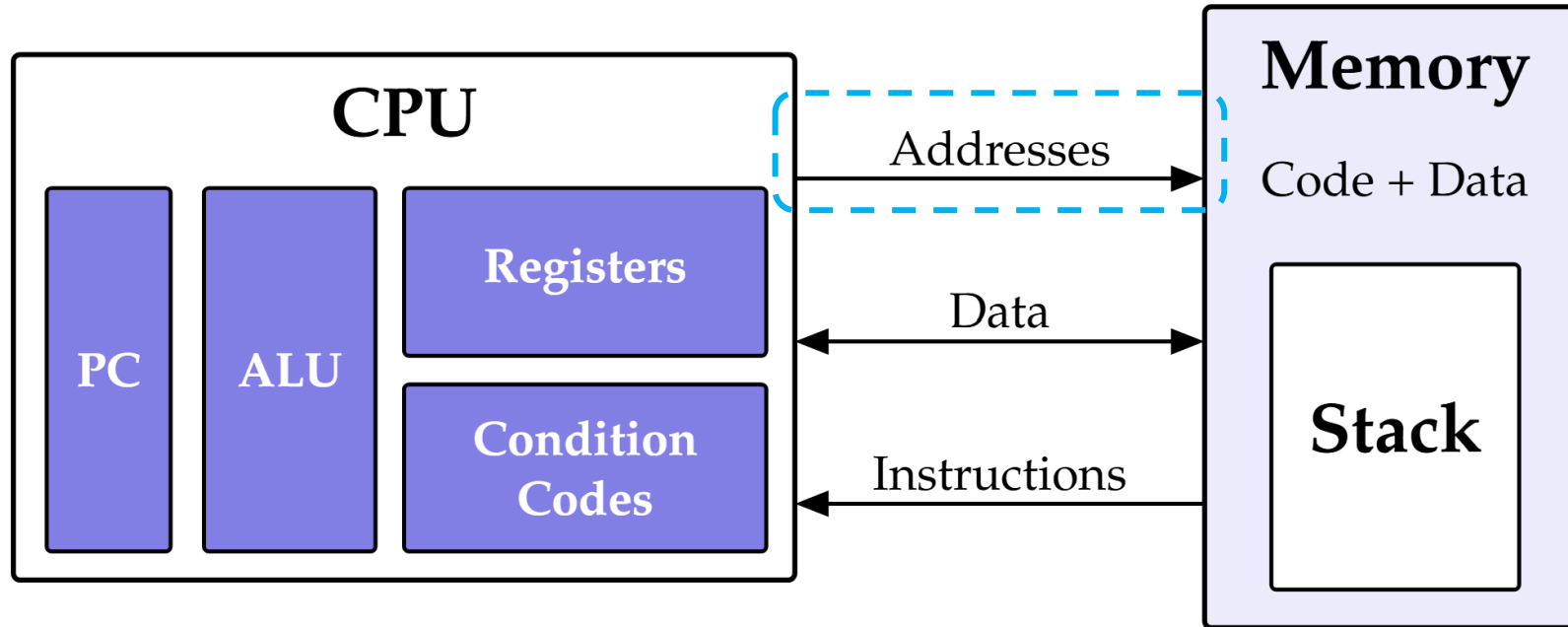
The PC holds the address of the **next instruction**.

- It tells the CPU where to fetch from.
- After a fetch, it usually moves to the next address.
- During a jump, call, or return, it is updated to a new address.

This is why the PC naturally drives the **address bus**.

Main Components

Breaking down the parts of a computer



Address Bus

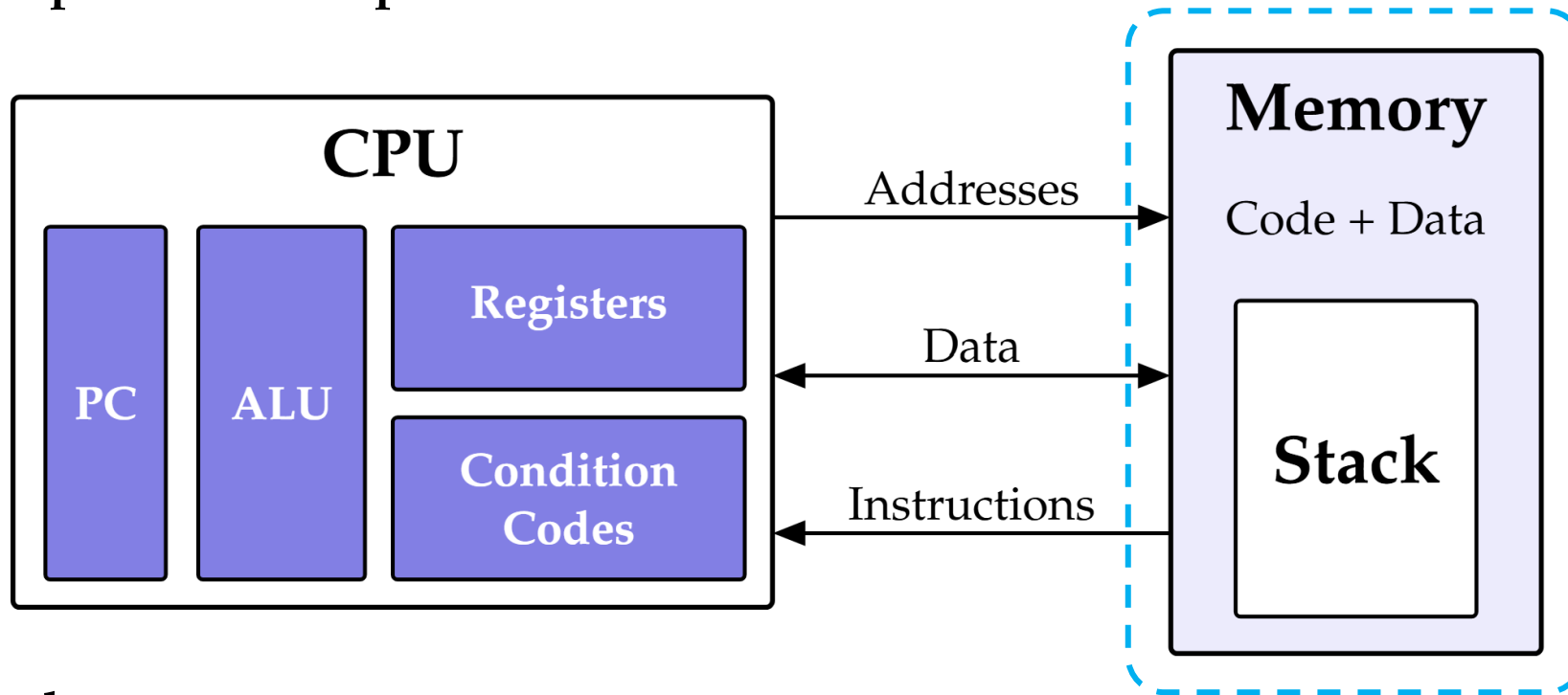
The address bus carries **location** information.

- It goes from CPU to memory.
- The CPU selects which memory location is being accessed.
- Memory does not send addresses back to the CPU in this basic model.

So, the address bus is **unidirectional**.

Main Components

Breaking down the parts of a computer



Memory and Stack

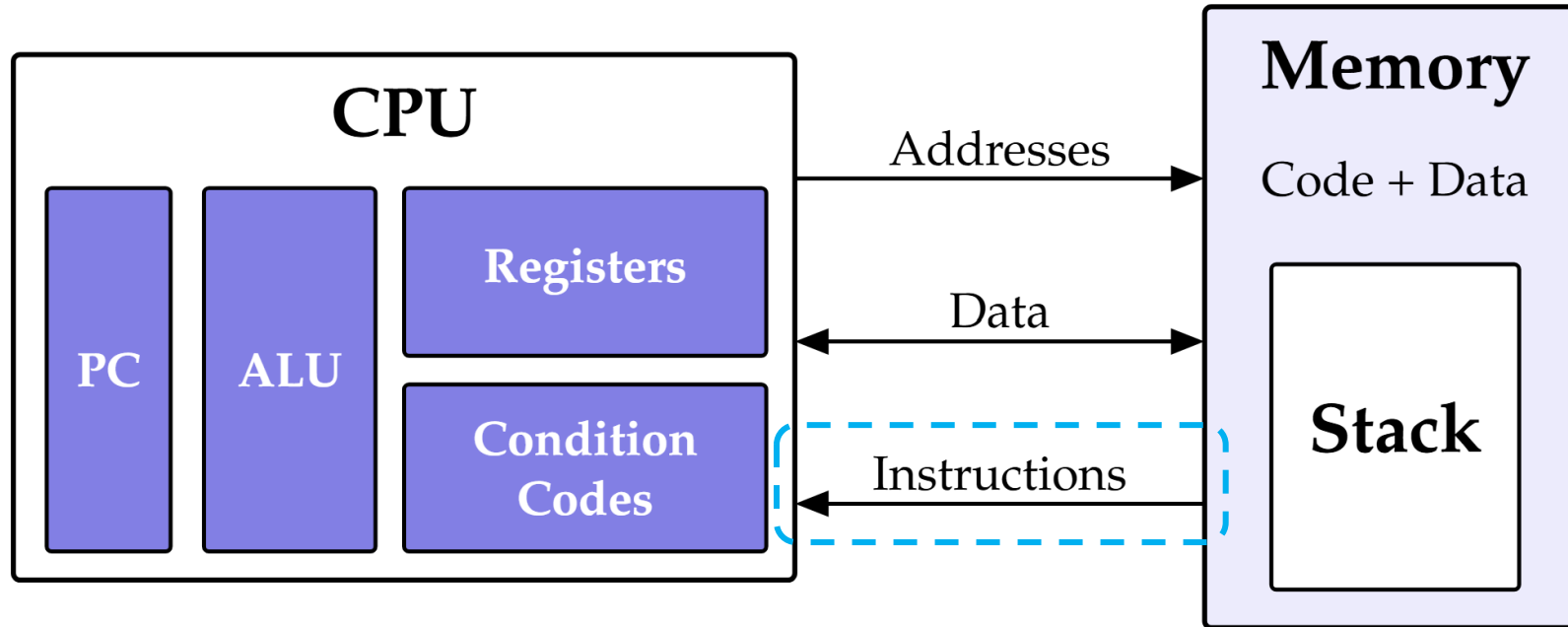
Memory stores both **instructions** and **data**.

- **Code** holds the program to be executed.
- **Data** holds operands and results.
- The **stack** is a special memory region used for temporary storage.

The stack is especially important for **CALL**, **RET**, interrupts, and saving values.

Main Components

Breaking down the parts of a computer



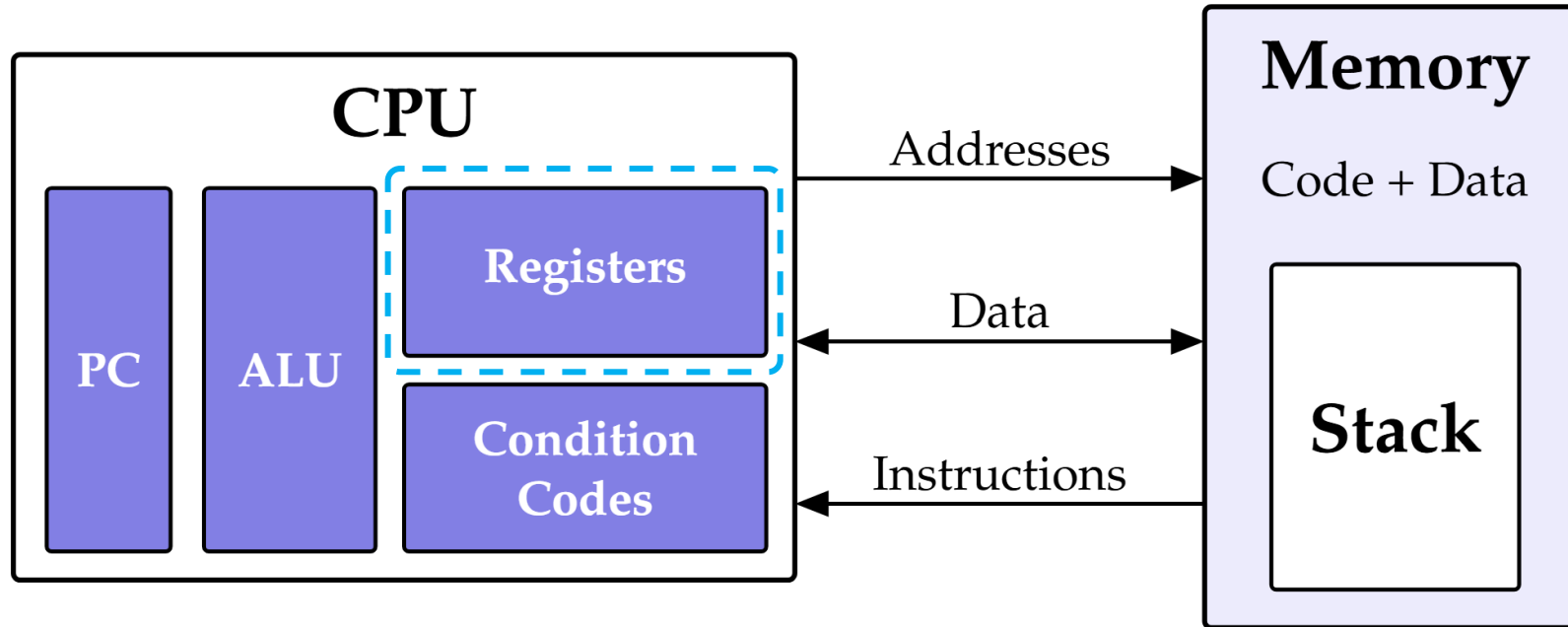
Instruction Flow (Data Bus)

The processor fetches instructions **from memory into the CPU**.

- This flow is shown as **memory** → **CPU**.
- It is **unidirectional** because fetching is a **read-only** operation.
- The CPU does not send instructions back to memory during execution.

Main Components

Breaking down the parts of a computer



Registers

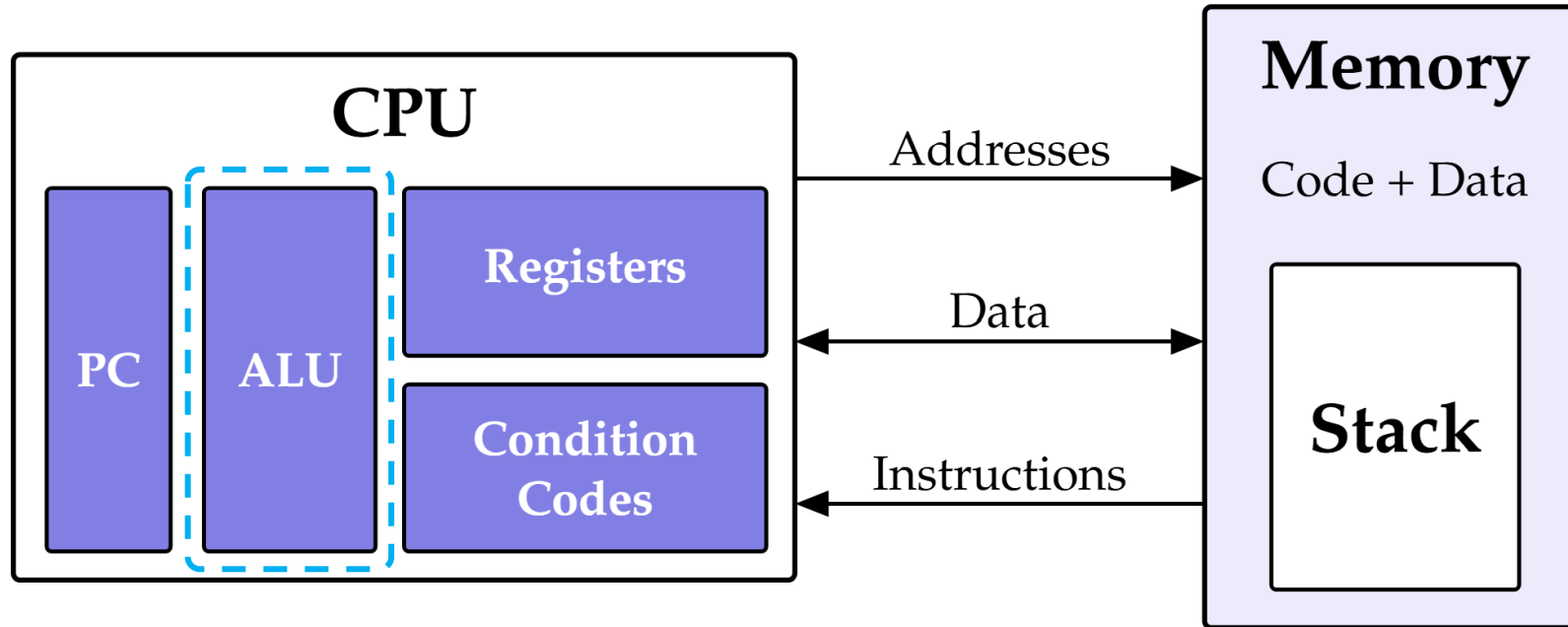
Registers are the CPU's **smallest** and **fastest** storage locations.

- They hold **operands**, **addresses**, and **intermediate results**.
- They reduce repeated access to slower main memory.
- Most ALU operations work on values already present in registers.

In assembly language, registers are central to efficient programming.

Main Components

Breaking down the parts of a computer



Arithmetic Logic Unit (ALU)

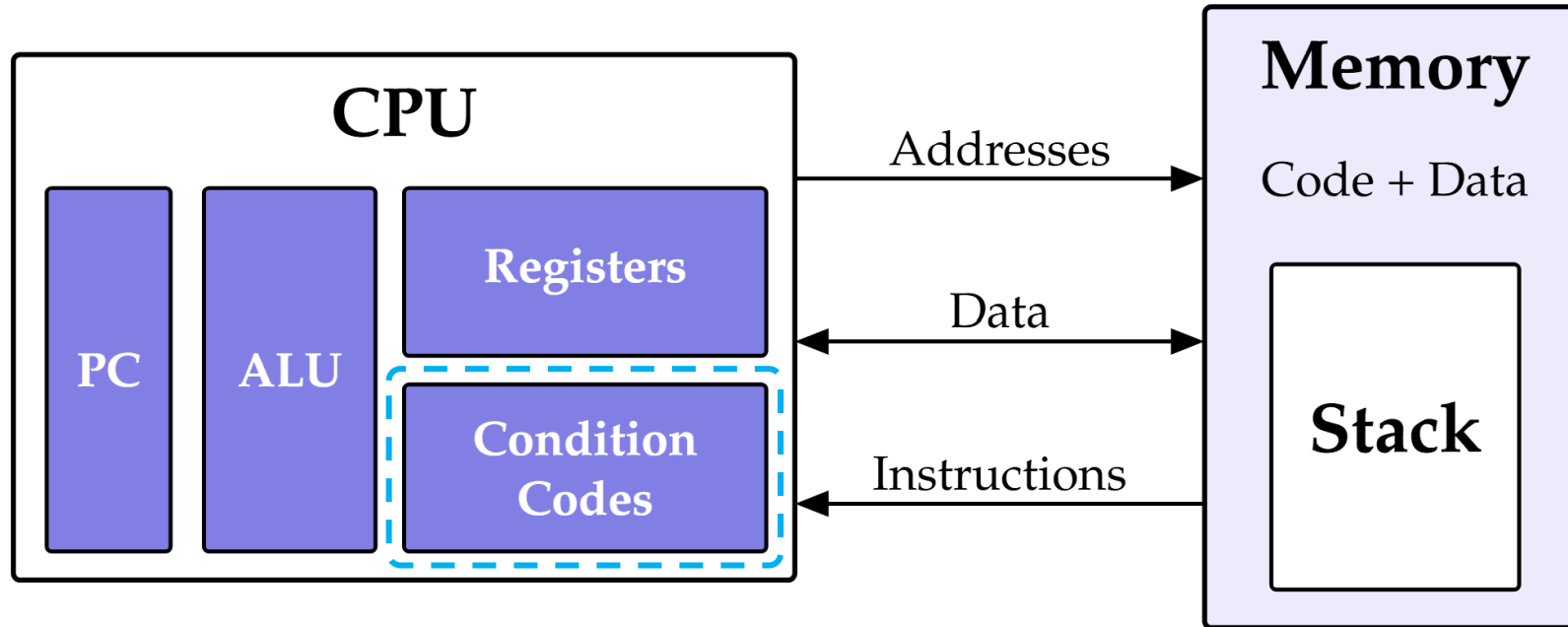
The Arithmetic Logic Unit performs mathematical and logical processing.

- **Arithmetic:** add, subtract, increment, decrement
- **Logic:** AND, OR, XOR, compare
- **Bit-level** manipulation and **decision** support

This is the part of the CPU responsible for **data processing**.

Main Components

Breaking down the parts of a computer



Condition Codes

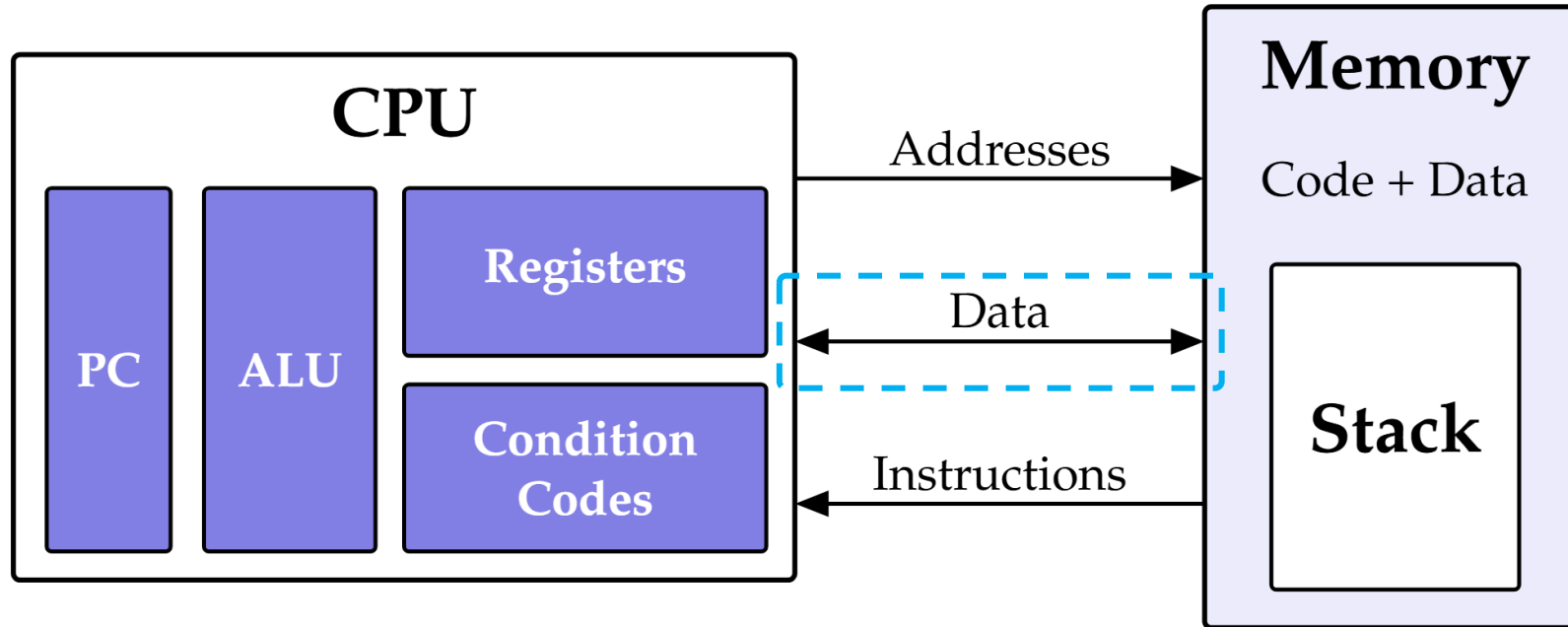
Condition codes, or **flags**, record the outcome of ALU operations.

- examples: zero, carry, sign, parity
- they tell the CPU what happened after an operation
- conditional jumps use them for decision making

They connect processing to program control.

Main Components

Breaking down the parts of a computer



Data Bus

The data bus carries actual **values**, so it must work in both directions.

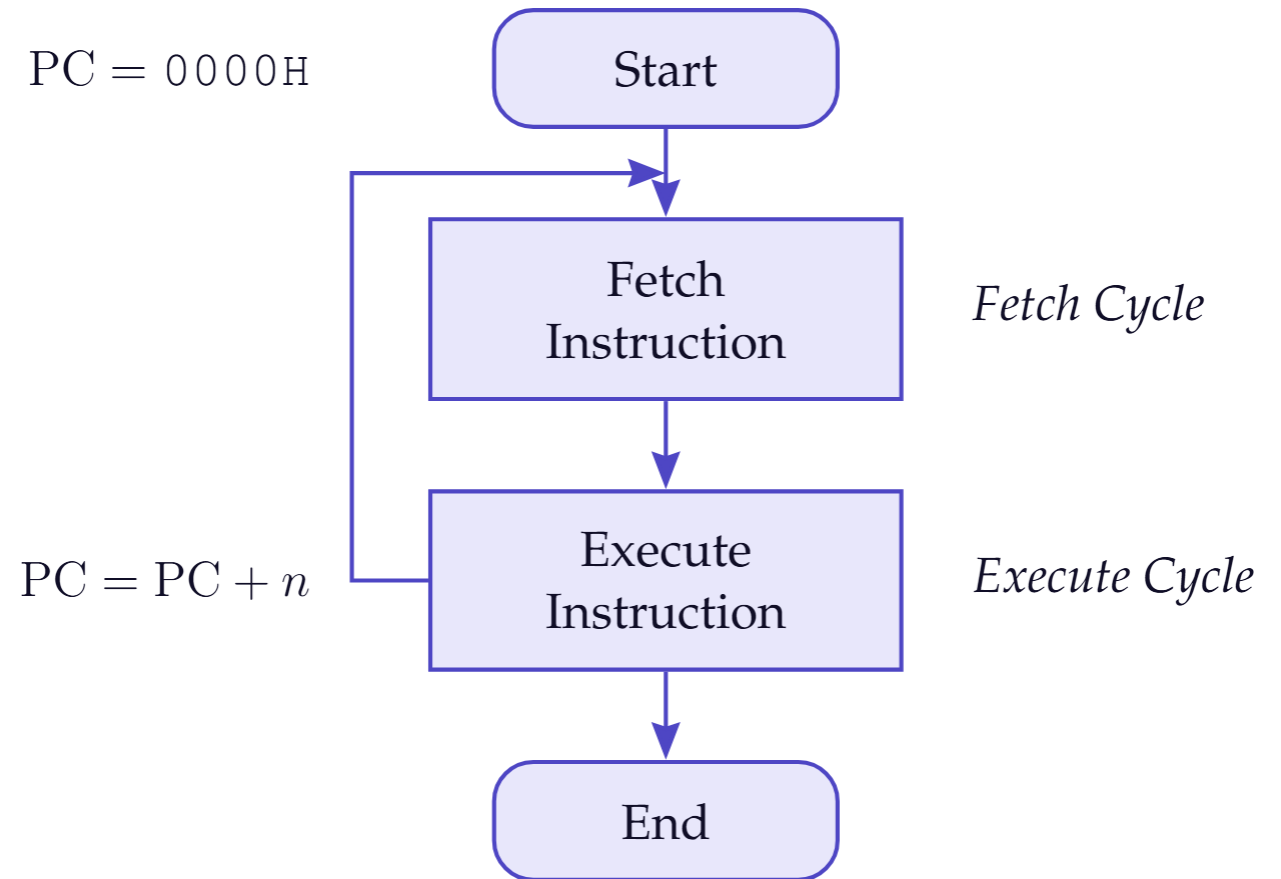
- **memory** → **CPU**: when the processor reads data
- **CPU** → **memory**: when the processor writes results

That is why the data bus is **bidirectional**, unlike the address bus and the instruction-fetch path.

Instruction Cycle

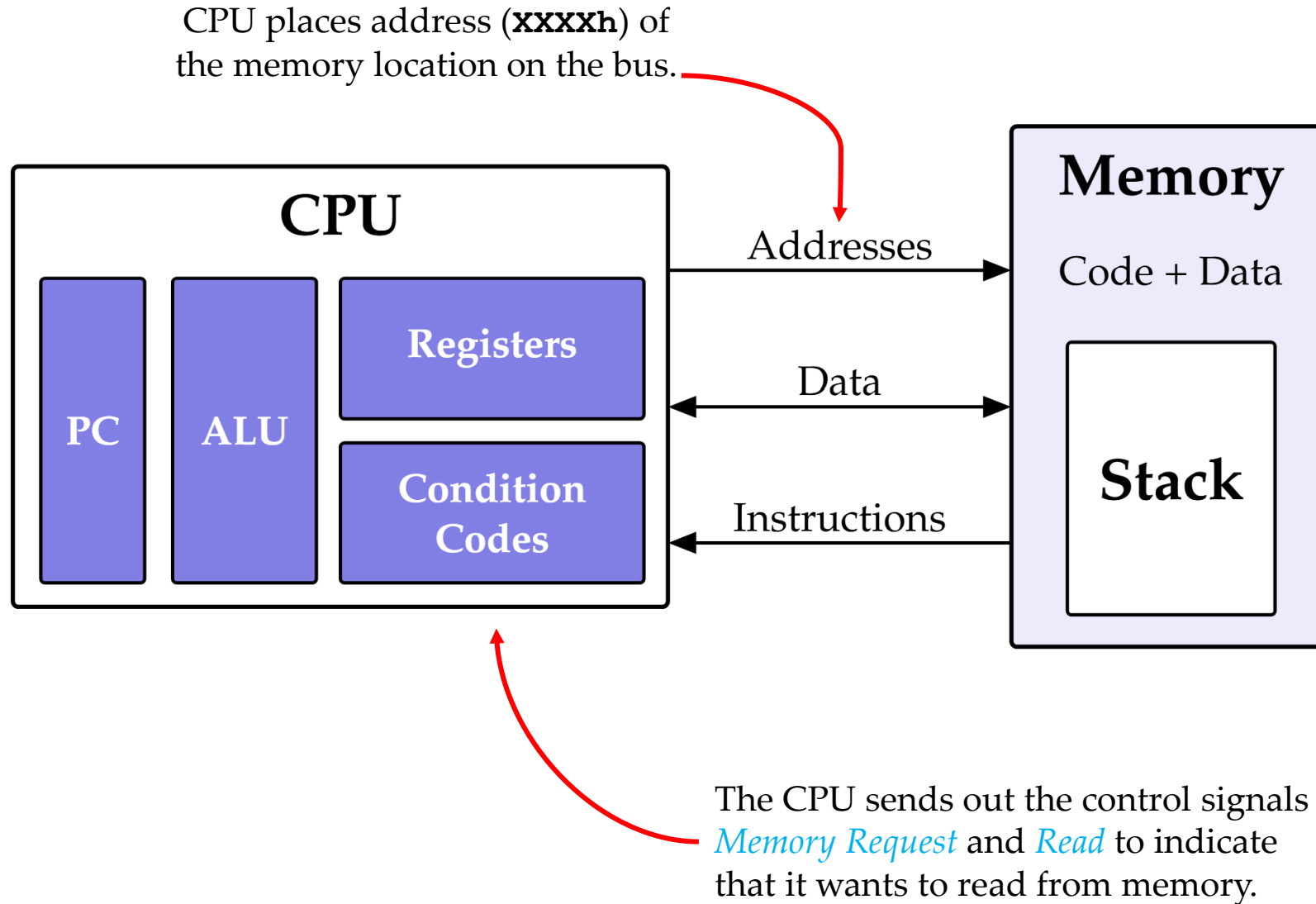
Fetch–Execute Loop

- **PC** holds the address of the next instruction.
- **Fetch:** read instruction from memory at address PC.
- **Execute:** decode + perform the operation (ALU/memory/branch).
- **Update PC:** typically, $PC \leftarrow PC + n$ (instruction length); branches/returns override this.
- Repeat until **HALT/exception/interrupt** ends or diverts the flow.



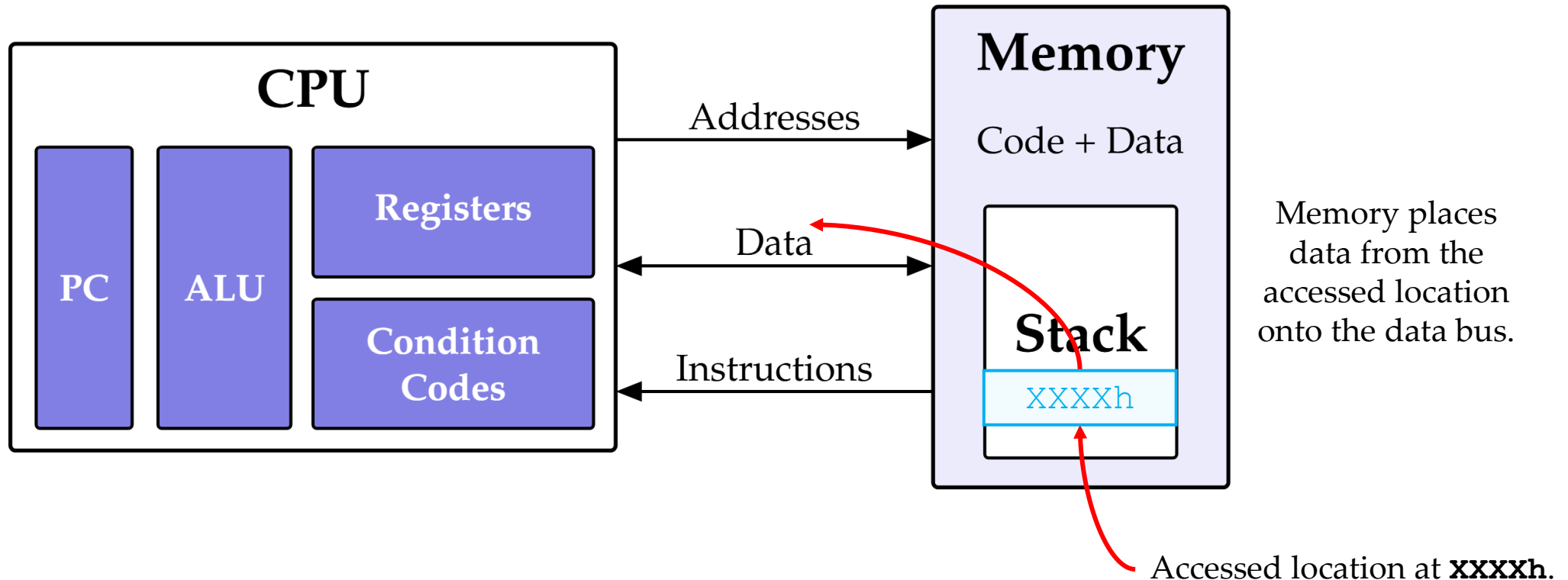
Read Operations

A Visual Simulation



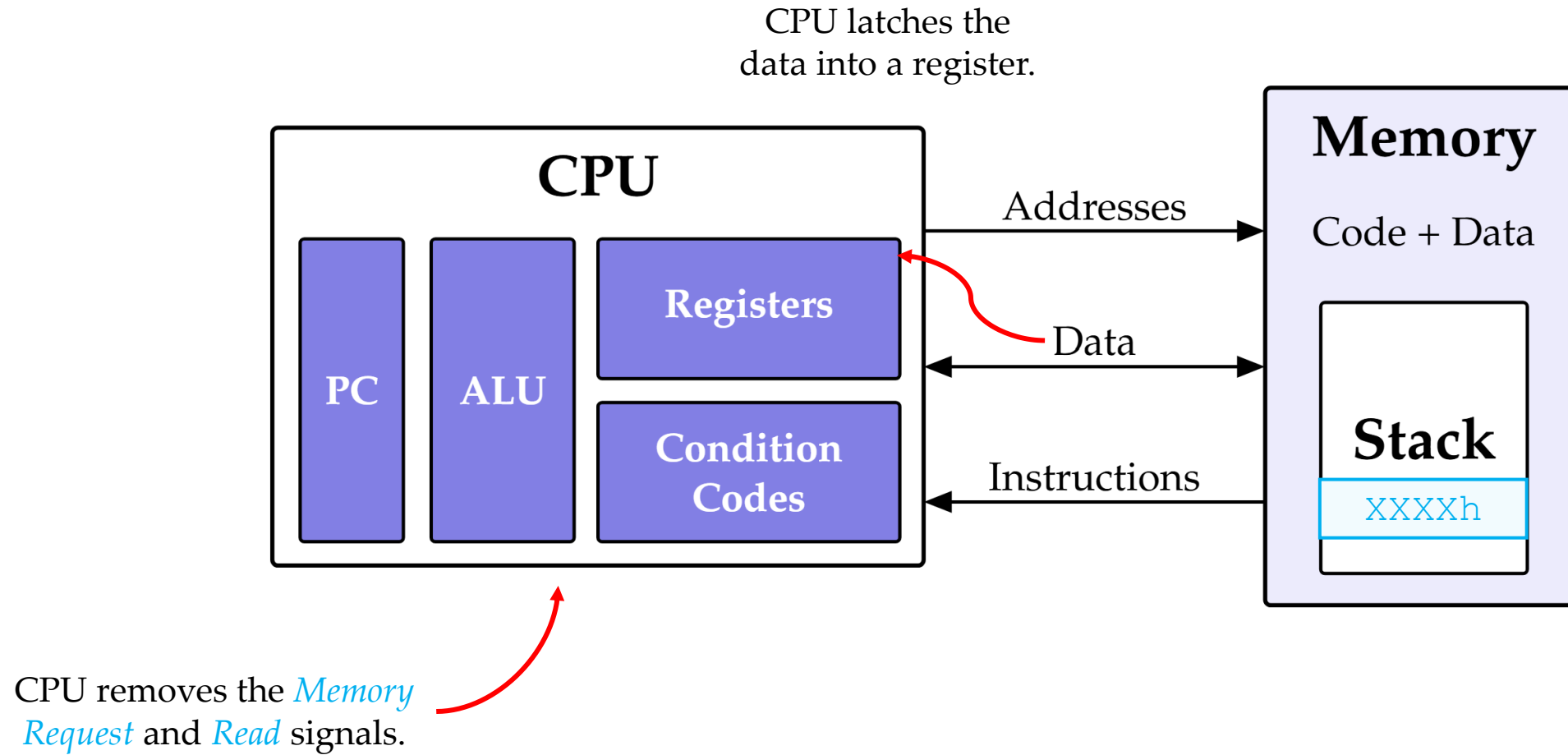
Read Operations

A Visual Simulation



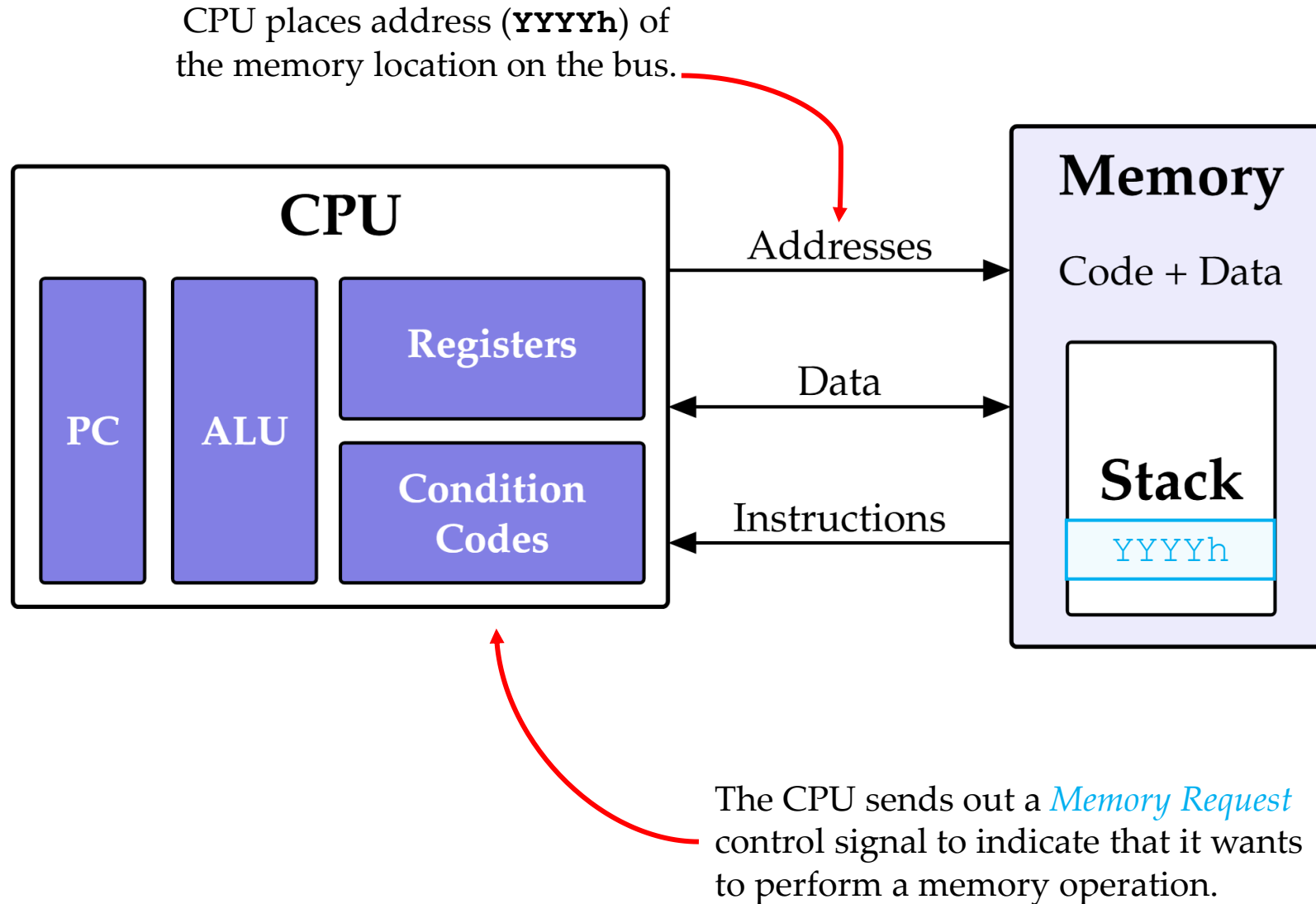
Read Operations

A Visual Simulation



Write Operations

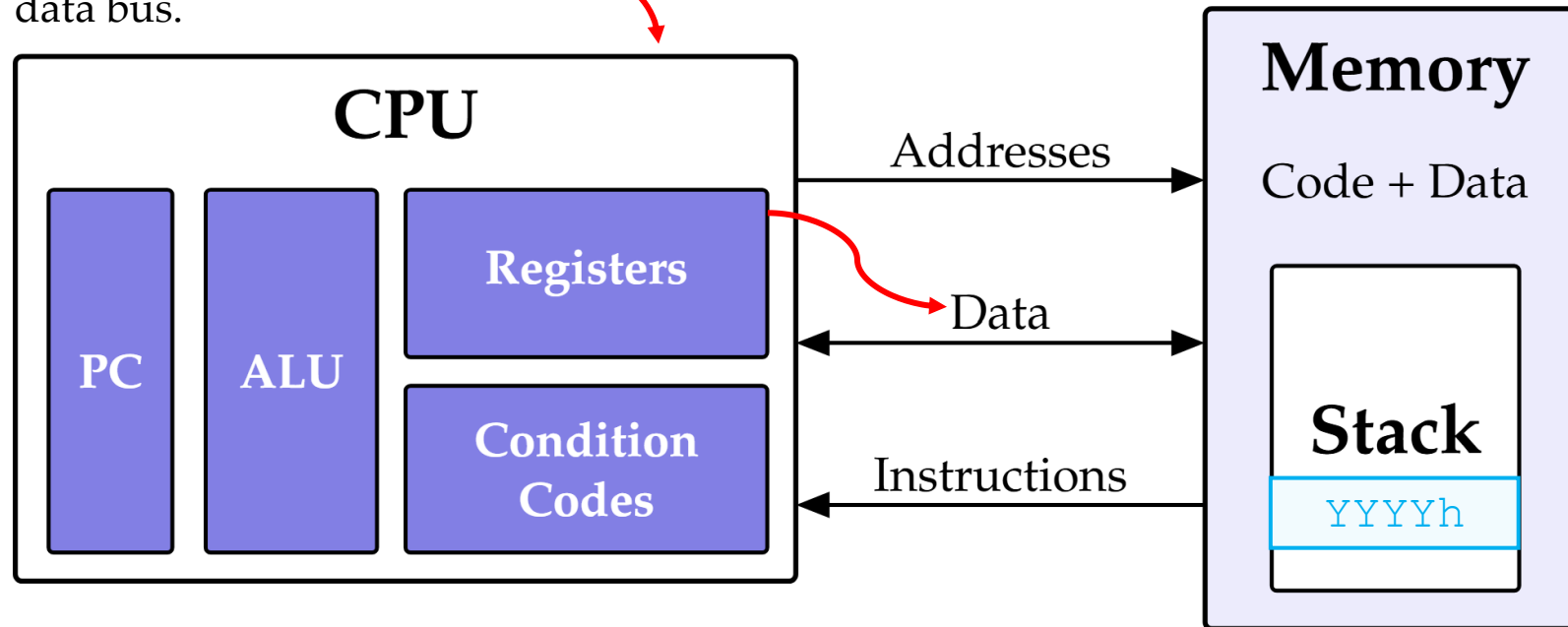
A Visual Simulation



Write Operations

A Visual Simulation

The CPU sends out a *Write* control signal to indicate that valid data is available on the data bus.

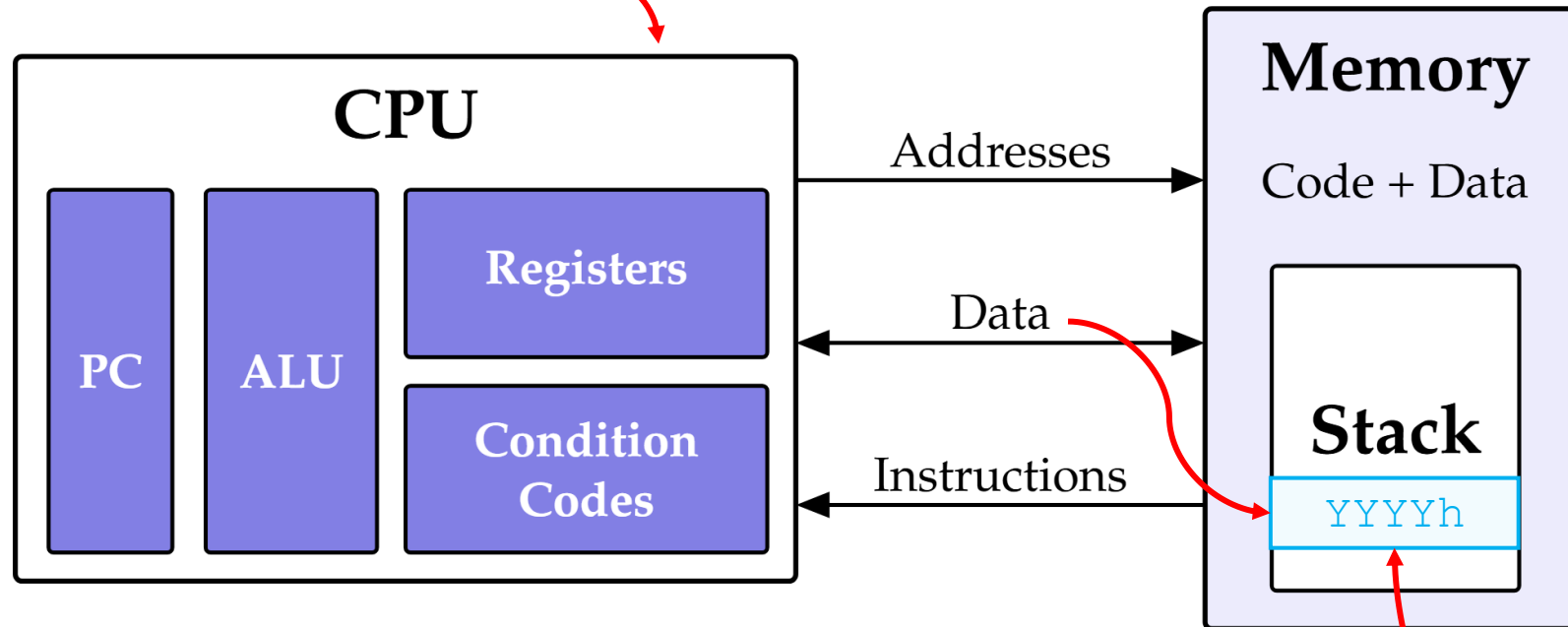


CPU places data from a register onto the data bus.

Write Operations

A Visual Simulation

The CPU removes the *Write* control signal to complete the memory write operation.



Memory copies the data bus into the accessed location.

Accessed location at **YYYYh**.

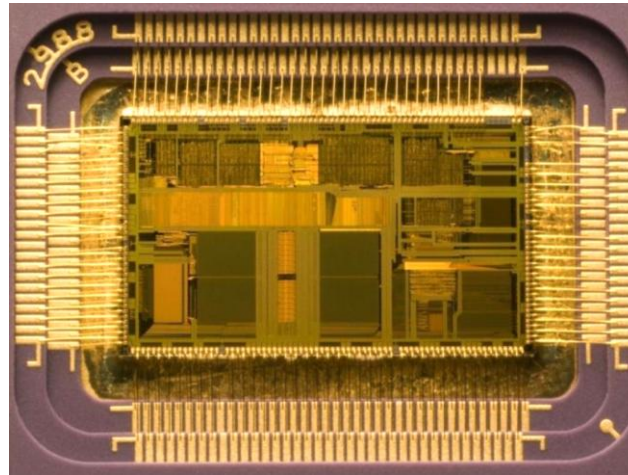
Microprocessor

What is it?

A *microprocessor* (also referred to as *μprocessor*) is a computer processor for which the data processing logic and control is included on a single **Integrated Circuit (IC)**.

It contains the *arithmetic*, *logic*, and *control* circuitry required to perform the functions of a computer's **Central Processing Unit (CPU)**.

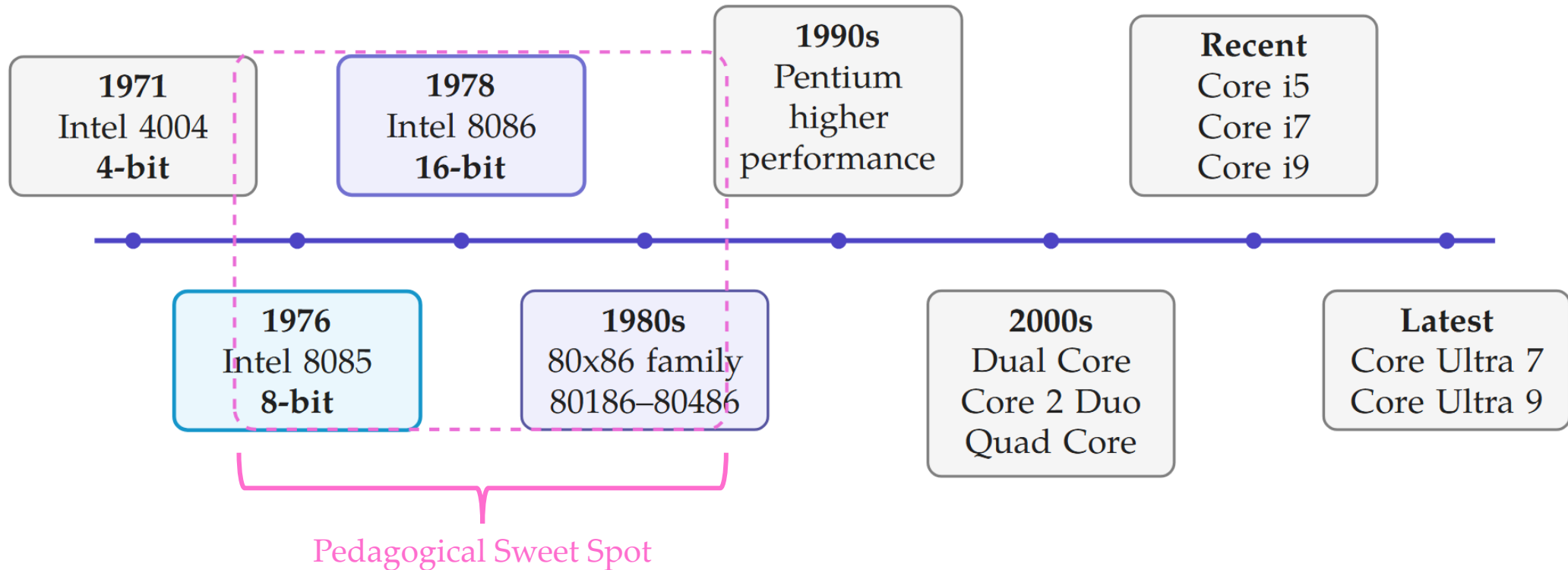
The integration of a whole CPU onto a single or a few integrated circuits using **Very-Large-Scale Integration (VLSI)** greatly reduced the cost of processing power.



Intel 80486DX2 microprocessor (actual size: 12 × 6.75 mm)

Evolution of Microprocessors

Chronological Development

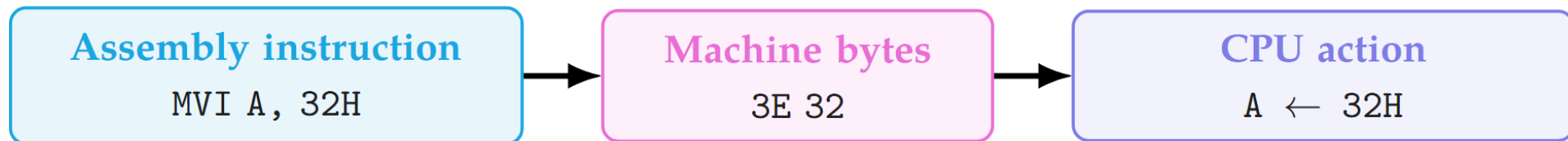


Assembly Language

What is it?

Assembly language is a *low-level programming language* with a very strong correspondence between the instructions in the language and the architecture's machine code instructions.

- Uses a *mnemonic* to represent low-level machine instructions (opcodes), directives, and usually architectural registers and flags. (e.g., **MVI**, **MOV**, **ADD**, **SUB**, **JMP**, and **JNZ**.)
- An *assembler* translates these mnemonics into machine-code bytes; the *processor* then performs the **fetch–decode–execute** cycle.



Assembly Language

An example for Intel 8085

How to read each line:

- **Address** tells where the instruction is stored in memory,
- **Mnemonic** is the human-readable assembly instruction,
- **Hex Code** is the actual machine-byte encoding executed by the 8085,
- and the comment explains the effect of the instruction.

	; Address	Mnemonic	Hex Code	; Meaning
➡	202A:	MVI A, 32H	3E 32	; Load 32H into accumulator A
➡	202C:	MVI B, 48H	06 48	; Load 48H into register B
➡	202E:	ADD B	80	; A <- A + B
➡	202F:	STA 41FFH	32 FF 41	; Store accumulator contents at memory location 41FFH
➡	2032:	HLT	76	; Stop program execution

Example task: Load two hexadecimal numbers into registers, add them, store the result in memory, and halt.

Assembly Language

Another example for Intel 8085

How this program behaves:

- First **24H** is loaded into register **B**; then **B** is incremented to **25H**. The updated value is copied into accumulator **A**. Next, **SUB B** subtracts **B** from **A**, so the result becomes **00H**. That final value is stored at memory location **5FFFH**.

	; Address	Mnemonic	Hex Code	; Meaning
➡	2020:	MVI B, 24H	06 24	; Load 24H into register B
➡	2022:	INR B	04	; Increment register B by 1
➡	2023:	MOV A, B	78	; Copy register B into accumulator A
➡	2024:	SUB B	90	; A ← A - B
➡	2025:	STA 5FFFH	32 FF 5F	; Store accumulator contents at memory location 5FFFH
➡	2028:	HLT	76	; Stop program execution

Example task: Load a value into register **B**, increment it, copy it into the accumulator, subtract **B** from the accumulator, store the result in memory, and halt.

Assembly Language

Try it yourself!

MVI A , 24H		<i>Load accumulator A with 24H</i>
MVI B , 56H	→	<i>Load register B with 56H</i>
ADD B	→	<i>ACC ← ACC + B</i>
OUT 01H	→	<i>Send accumulator contents to port 01H</i>
HLT	→	<i>End the program</i>

What value will be displayed/output at port **01H**?

$$\mathbf{24H + 56H = 7AH}$$

So, after **ADD B**, the accumulator contains **7AH**. Therefore, **OUT 01H** sends **7AH** to port **01H**.

Assembly Language

An example for Intel 8086

How this program behaves:

- The first input is read into **AL** and then copied into **BL**. A second input is again read into **AL**. The instruction **ADD AL, BL** adds the saved first number to the second number, leaving the sum in **AL**. Finally, **OUT 30H, AL** sends that sum to output port **30H**.

	; Address	Mnemonic	Hex Code	; Meaning
➔	0100:	IN AL, 27H	E4 27	; Input first number from port 27H into AL
➔	0102:	MOV BL, AL	88 C3	; Save a copy of AL in register BL
➔	0104:	IN AL, 27H	E4 27	; Input second number from port 27H into AL
➔	0106:	ADD AL, BL	00 D8	; Add BL to AL and store the sum in AL
➔	0108:	OUT 30H, AL	E6 30	; Output AL to port 30H
➔	010A:	HLT	F4	; Halt the computer

Example task: Input two numbers, add them, output the sum to a port, and halt.